Code Explanation

Code Explanation: AWS Glue PySpark ETL JobThis code implements a production-ready AWS Glue ETL job that processes customer and order data with data quality controls, SCD Type 2 tracking, and aggregations.---

High-Level Flow

1. **Initialize** Spark/Glue contexts and load configuration2. **Ingest** customer and order CSV data from S33. **Clean** data by removing nulls and duplicates4. **Catalog** cleaned data in AWS Glue Data Catalog5. **Implement SCD Type 2** for order history using Apache Hudi6. **Aggregate** customer spending metrics7. **Register** all output tables in the Glue Catalog---

Component Breakdown

1. SparkContextManager

Purpose: Initialize Spark and Glue execution contexts**Key Method:** `initialize_spark_contexts()`- Detects if running in AWS Glue environment or locally- Creates SparkSession with Hudi extensions enabled- Returns spark, glueContext, and job objects for downstream use---

2. ConfigManager

Purpose: Load job parameters from YAML config or Glue job arguments**Key Methods:**- `load_config()` - Reads `config/glue_params.yaml` file- `get_job_parameters()` - Merges Glue arguments with config file, applies defaults**Default Parameters Include:**- S3 paths for source/target data- Glue database and table names- CSV parsing options (delimiter, header)- Hudi configuration (record key, precombine field, table type)---

3. S3DataReader

Purpose: Read data from S3 with error handling**Key Method:** `read_data_safe()`- Supports CSV and Parquet formats- Normalizes column names to lowercase- Logs record counts- Returns DataFrame or raises exception on failure**CSV Reading Options:**- Header detection- Custom delimiter support- Quote/escape character handling- Schema inference---

4. S3DataWriter

Purpose: Write data to S3 in multiple formats**Key Method:** `write_data_safe()`- Supports Parquet and Hudi formats- Configurable write modes (overwrite/append)- Validates Hudi options before writing- Logs success/failure with detailed error messages---

5. DataCleaner

Purpose: Apply data quality rules**Key Method:** `remove_nulls_and_duplicates()`- Removes rows with actual NULL values- Removes rows with string "Null" or "null" values- Drops duplicate records- Logs before/after record counts---

6. CatalogManager

Purpose: Register tables in AWS Glue Data Catalog**Key Method:** `register_table()`- Creates database if it doesn't exist- Drops existing table to avoid conflicts- Creates external table pointing to S3 location- Uses Parquet format for catalog tables---

7. HudiManager

Purpose: Implement SCD Type 2 using Apache Hudi**Key Methods:****`prepare_scd_type2_data()`**- Adds SCD Type 2 columns: - `IsActive` - Boolean flag (all set to True for new records) - `StartDate` - Timestamp when record became active - `EndDate` - Timestamp when record was superseded (NULL for active) - `OpTs` - Operation timestamp for Hudi precombine logic**`get_hudi_options()`**- Configures Hudi table properties: - Record key field (unique identifier) - Precombine field (for conflict resolution) - Table type (COPY_ON_WRITE for read-optimized performance) - Operation type (upsert for merge logic) - Parallelism settings---

8. AggregationEngine

Purpose: Calculate business metrics**Key Method:** `calculate_customer_aggregate_spend()`- Calculates line total: `PricePerUnit × Qty`- Groups by customer ID- Sums all line totals per customer- Returns DataFrame with `CustId` and `TotalSpend`---

Main Execution Flow

Step 1: Initialization

```
Initialize Spark contexts → Load parameters → Initialize all component classes
```

Step 2: Data Ingestion (TR-INGEST-001, TR-INGEST-002)

```
Read customer CSV from S3 → Validate schemaRead order CSV from S3 → Validate
schema
```

Schema Validation:- Customer: `custid, name, emailid, region`- Order: `orderid, itemname, priceperunit, qty, date, custid`

Step 3: Data Cleaning (TR-CLEAN-001, TR-CLEAN-002)

```
Remove nulls and duplicates from customer dataRemove nulls and duplicates from
order data
```

Step 4: Catalog Registration (TR-CATALOG-001, TR-CATALOG-002)

```
Write cleaned customer data to Parquet → Register in Glue CatalogWrite cleaned
order data to Parquet → Register in Glue Catalog
```

Step 5: SCD Type 2 Implementation

```
Add SCD columns to order data → Write to Hudi format → Register Hudi table
```

Hudi Benefits:- Tracks historical changes to order records- Supports time-travel queries- Enables incremental processing with upsert logic

Step 6: Aggregation (TR-AGG-001)

```
Calculate total spend per customer → Write to Parquet → Register in Glue Catalog
```

Step 7: Job Completion

```
Commit Glue job → Log success
```

Error Handling Strategy

1. **Graceful Degradation:** Each component logs errors before raising exceptions2. **Schema Validation:** Verifies expected columns exist before processing3. **Null Safety:** Checks for both NULL values and string "Null"4. **Path Validation:** Raises clear errors if S3 paths don't exist5. **Configuration Fallback:** Uses defaults if config file missing---

Key Design Patterns

1. **Separation of Concerns:** Each class handles one responsibility2. **Dependency Injection:** Components receive dependencies (spark, params) rather than creating them3. **Testability:** Internal methods (`_read_csv_internal`, `_write_parquet`) can be mocked4. **Logging:** Comprehensive logging at INFO and ERROR levels5. **Configuration Management:** Centralized parameter handling with defaults---

Output Artifacts

| Artifact | Format | Location | Purpose ||----------|--------|----------|---------|| customer_cleaned | Parquet | `<customer_source_path>/catalog/` | Cleaned customer master data || order_cleaned | Parquet | `<order_source_path>/catalog/` | Cleaned order transaction data || ordersummary | Hudi | `<ordersummary_target_path>` | Order history with SCD Type 2 || customeraggregatespend | Parquet | `<customeraggregatespend_target_path>` | Customer spending metrics |All tables are registered in the `gen_ai_poc_databrickscoe` Glue database.